

# Gestão de Equipamentos

API REST em .NET — Trabalho Final do Módulo 3 (Desenvolvimento em C#)

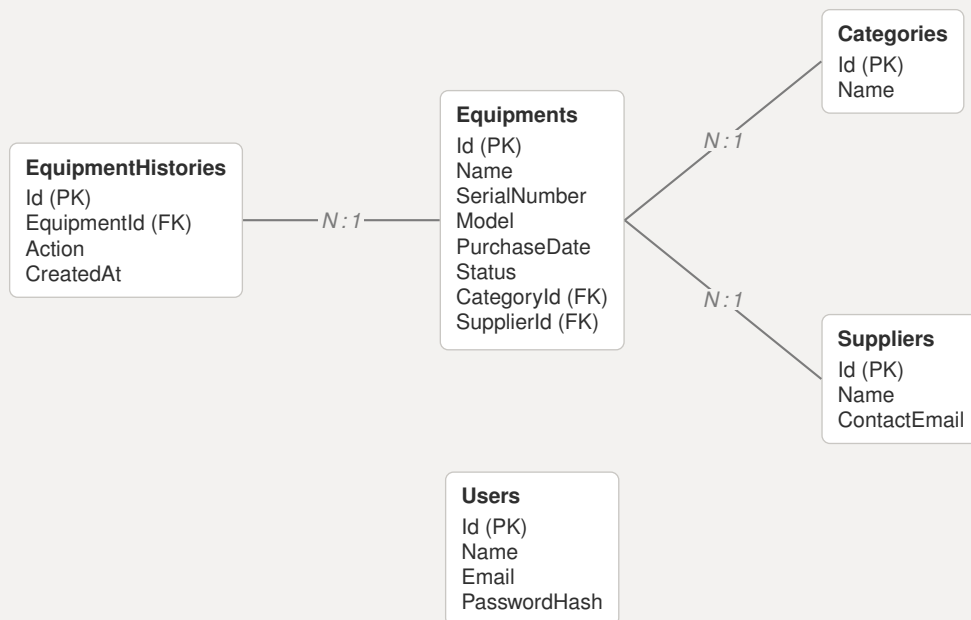
Documento de planejamento e status do projeto.

## 1 Resumo do projeto

Sistema de **Gestão de Equipamentos**: cadastro de equipamentos, categorias, localização (fornecedores), histórico de manutenções/ações e usuários. API REST em ASP.NET Core, persistência em PostgreSQL via Entity Framework Core, arquitetura em camadas (API, Application, Domain, Infrastructure, Exceptions).

## 2 Modelo de dados

Esquema gerado pelas entidades + migrations do EF Core e verificado no PostgreSQL. Relacionamentos por chave estrangeira em destaque.



Users ainda sem relacionamento (entra com a autenticação).

## Entidades

| Entidade                | Descrição                                                 |
|-------------------------|-----------------------------------------------------------|
| <b>Equipment</b>        | Item gerenciado; liga-se a Category e Supplier.           |
| <b>Category</b>         | Classificação do equipamento (nome único).                |
| <b>Supplier</b>         | Fornecedor (nome e e-mail de contato).                    |
| <b>EquipmentHistory</b> | Histórico de ações de um equipamento.                     |
| <b>User</b>             | Usuário do sistema (login/senha) — usado na autenticação. |

## 3 Arquitetura em camadas

```
Cliente / Swagger / Postman
|
GestaoEquipamentos.API          controllers, Swagger, auth, Program.cs
GestaoEquipamentos.Application  DTOs, services, interfaces, casos de uso
GestaoEquipamentos.Domain       entidades, enums, regras essenciais
GestaoEquipamentos.Infrastructure DbContext, repositories, configurations, migrations
|
PostgreSQL
```

A camada Domain não referencia EF Core nem infraestrutura. Exceptions é transversal (exceções customizadas e padronização de erros).

## 4 Divisão de tarefas

| Camada / Tema         | Responsável   | Status                                                                                           |
|-----------------------|---------------|--------------------------------------------------------------------------------------------------|
| <b>Infrastructure</b> | Camila        | <input checked="" type="checkbox"/> Concluída (repositórios, DbSet, configurations, migrations). |
| <b>Domain</b>         | Lucas         | <input checked="" type="checkbox"/> Entidades e enums criados.                                   |
| <b>API</b>            | Lucas / Pedro | <input type="checkbox"/> Controllers, Swagger, middlewares, wiring.                              |
| <b>Application</b>    | Paulo         | <input type="checkbox"/> DTOs, services e validações em andamento.                               |
| <b>Exceptions</b>     | Adriano       | <input type="checkbox"/> Exceções e middleware global de erro.                                   |
| <b>Autenticação</b>   | Pedro         | <input type="checkbox"/> Registro/login, hash de senha, JWT.                                     |

## 5 Camada Infrastructure — entregue

- **Repositório genérico** `Repository<T> + IRepository<T>` (CRUD reutilizável).
- **Repositórios específicos:** `Equipment` (com filtro por categoria), `Category`, `Supplier` e `EquipmentHistory` (com filtro por equipamento).
- **DbContext** com os `DbSet` de todas as entidades e aplicação automática das configurations (`ApplyConfigurationsFromAssembly`).
- **Configurations** (`IEntityTypeConfiguration`): tamanhos de campo, índice único (`CategoryName`, `Equipment.SerialNumber`), tipo `date` e a chave estrangeira `EquipmentHistory`

→ Equipment.

- **Injeção de dependência:** AddInfrastructure registra o DbContext (Npgsql/PostgreSQL) e todos os repositórios.
- **Banco:** PostgreSQL em container (porta 5433), connection string configurada.

## 6 Artefatos

### Migrations aplicadas

| Migration                     | O que faz                                                 |
|-------------------------------|-----------------------------------------------------------|
| InitialCreate                 | Cria todas as tabelas a partir das entidades.             |
| ConfigureEquipment            | Tamanhos de campo, serial único e PurchaseDate como date. |
| ConfigureRemainingEntities    | Regras de Category, Supplier e EquipmentHistory.          |
| AddEquipmentHistoryForeignKey | FK EquipmentHistory → Equipment.                          |

### Como rodar (resumo)

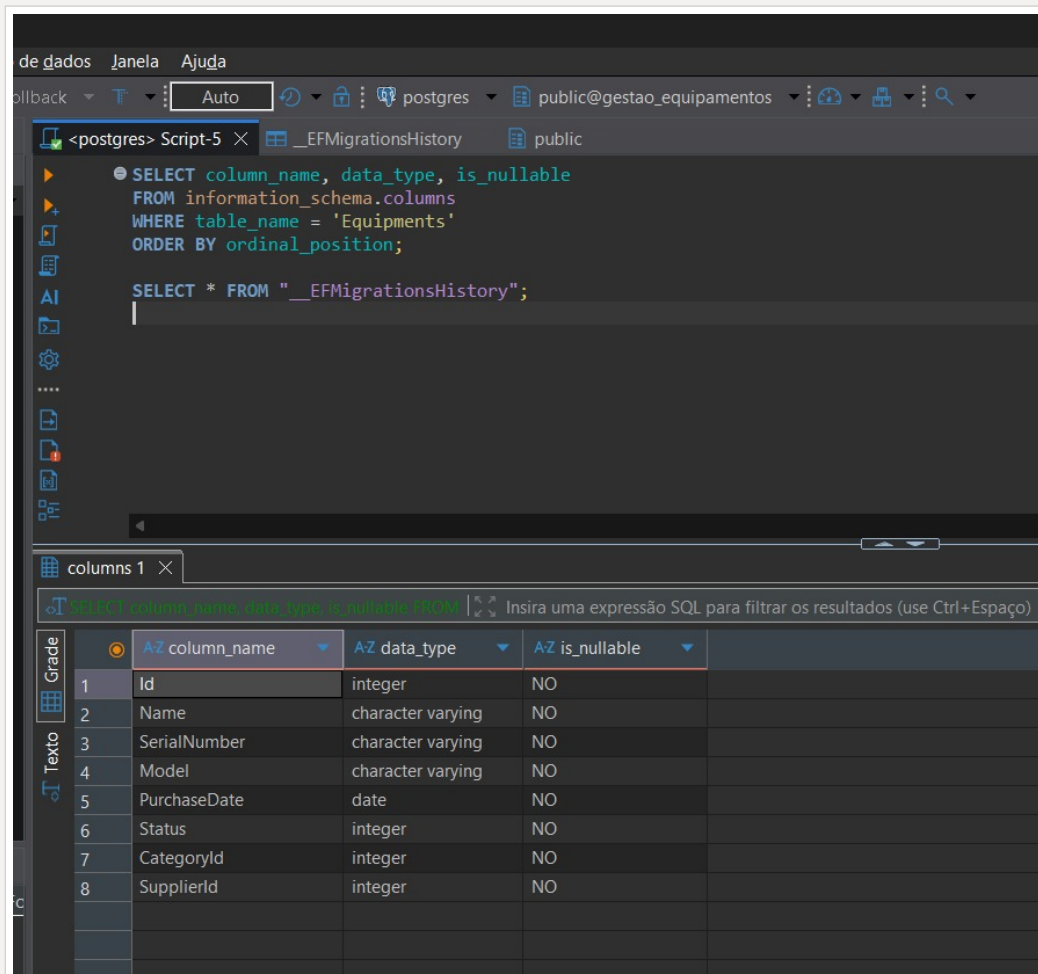
```
docker compose up -d
dotnet ef database update --project GestaoEquipamentos.Infrastructure
dotnet run --project GestaoEquipamentos.API
```

## 7 Evidências (PostgreSQL via DBeaver)

Banco gestao\_equipamentos inspecionado no DBeaver após aplicar as migrations.



Tabela \_\_EFMigrationsHistory — as 4 migrations aplicadas no banco.



Colunas da tabela Equipments — tipos aplicados pela configuration (varchar, date).

## 8 Checklist do professor

- |                                                           |                                                                      |
|-----------------------------------------------------------|----------------------------------------------------------------------|
| <input checked="" type="checkbox"/> PostgreSQL como banco | <input checked="" type="checkbox"/> EF Core com provider Npgsql      |
| <input checked="" type="checkbox"/> Solução em 5 projetos | <input checked="" type="checkbox"/> Acesso a dados na Infrastructure |
| <input checked="" type="checkbox"/> Migrations entregues  | <input checked="" type="checkbox"/> Relacionamentos entre entidades  |
| <input checked="" type="checkbox"/> Repositório Git       | <input type="checkbox"/> Autenticação (JWT)                          |
| <input type="checkbox"/> DTOs de entrada/saída            | <input type="checkbox"/> Regras de negócio em services               |
| <input type="checkbox"/> Swagger habilitado               | <input type="checkbox"/> Endpoints (auth, CRUD, filtros)             |

Legenda: ☒ concluído   ☐ pendente.